

PRÁCTICA 2

Control de una puerta de garaje

DOCUMENTO	ASIGNATURA	TITULACIÓN	CENTRO
Del alumnado	**Arq. de Computadores II**	**Grado en Ing. Informática · 2.º**	**ESEI · Universidade de Vigo**

Contenido

1. Ficha de la práctica 4

2. Objetivos de aprendizaje 4

3. El sistema que hay que construir 5

4. Modelo del sistema 5

4.1 Recorrido, velocidad y posición 5

4.2 Entradas 6

4.3 Salidas 7

5. Comportamiento exigido 7

5.1 Estados 7

5.2 Transiciones 8

5.3 La fotocélula 9

6. Desarrollo paso a paso 9

Paso 1. Proyecto y biblioteca 9

Paso 2. Base de tiempo de 100 ms 10

Paso 3. Flanco del mando, con antirrebote no bloqueante 11

Paso 4. La máquina de estados 11

Paso 5. Visualización 12

Paso 6. Mira la máquina de estados moverse 13

7. Criterios de aceptación 13

8. Criterios de evaluación 14

9. Entregables 16

10. Uso de asistentes de inteligencia artificial 16

Anexo A. La biblioteca TM1638 17

A.1 Inicialización 17

A.2 Funciones 17

Anexo B. Conexionado 18

Anexo C. El protocolo del TM1638 20

1. Ficha de la práctica

CAMPO	VALOR
Sesión	Sesiones 2 y 3 de 13
Duración	4 horas presenciales, en dos sesiones
Modalidad	Individual, en el laboratorio
Requisitos previos	Práctica 1 superada: crear un proyecto en STM32CubeIDE, configurar GPIO y compilar
Material	Placa NUCLEO-F401RE · módulo LED&KEY (TM1638) · STM32CubeIDE · biblioteca TM1638.c / TM1638.h
Entregables	Proyecto CubeIDE comprimido, diagrama de estados y memoria breve (ver apartado 9)
Plazo	Una semana desde el final de la segunda sesión, por Moovi

2. Objetivos de aprendizaje

Al terminar esta práctica serás capaz de:

- Traducir un enunciado en lenguaje natural a una **máquina de estados finitos** explícita, con sus estados, transiciones y acciones separadas.
- Escribir un bucle de control **no bloqueante**: el programa atiende todas las entradas en todo momento, sin quedarse esperando dentro de una espera.
- Detectar **flancos** en una entrada digital y filtrar los rebotes mecánicos de un pulsador sin detener el programa.

- Medir tiempo con **HAL_GetTick()** y construir una base de tiempo periódica que no acumule deriva.
- Gobernar los LEDs y el display de siete segmentos de un módulo TM1638 desde una biblioteca ya escrita.

> Esta práctica no va de mover una puerta. Va de **separar el estado del sistema de la lógica que lo cambia**. Es el mismo patrón que encontrarás en un controlador de ascensor, en un cajero o en el arranque de un motor industrial.

3. El sistema que hay que construir

Un automatismo de puerta de garaje tiene que resolver tres cosas a la vez: saber en qué situación está la puerta, reaccionar al mando a distancia y no aplastar a nadie. Vas a construir ese automatismo completo, sustituyendo el motor y los sensores reales por el módulo LED&KEY.

La puerta puede estar cerrada, abierta, moviéndose en cualquiera de los dos sentidos o detenida a mitad de camino. El mando a distancia es un único pulsador: no dice "abre" ni "cierra", solo dice "cambia". Qué signifique ese "cambia" depende de lo que la puerta estuviera haciendo, y ahí está el problema que resuelve la máquina de estados.

4. Modelo del sistema

Todo lo que sigue es de obligado cumplimiento. Los valores concretos importan: los criterios de aceptación del apartado 7 se comprueban con ellos.

4.1 Recorrido, velocidad y posición

MAGNITUD	VALOR	CONSTANTE EN EL CÓDIGO
Recorrido total de la hoja	300 cm (3,00 m)	RECORRIDO_CM

MAGNITUD	VALOR	CONSTANTE EN EL CÓDIGO
Avance en cada paso	3 cm	PASO_CM
Periodo del paso	100 ms	TICK_MOV_MS
Velocidad resultante	30 cm/s	—
Duración del recorrido completo	10,0 s	—

La variable **posicion** guarda la posición de la hoja en centímetros: vale **0 con la puerta cerrada** y **300 con la puerta completamente abierta**. Se actualiza únicamente cuando el motor está en marcha, sumando o restando PASO_CM una vez por cada periodo de 100 ms.

> Ojo con la unidad. La posición se lleva **en centímetros**, no en "número de LED". Los ocho LEDs son una representación de la posición, no la posición misma. Confundirlos es el error más frecuente en esta práctica.

4.2 Entradas

ENTRADA	DISPOSITIVO FÍSICO	PAPEL EN EL SISTEMA
MANDO	Pulsador azul de usuario de la NUCLEO (PC13). Activo a nivel bajo.	Mando a distancia. Cada pulsación es una orden de "cambia lo que estés haciendo".
FOTOCÉLULA	Tecla **S1** del módulo LED&KEY.	Detector de presencia. Mientras esté pulsada hay un obstáculo bajo la puerta.

> **Cuidado al cablear: estas dos entradas se llaman MANDO y FOTOCÉLULA, no B0 y B1.** En el serigrafiado de la placa **B1 es el pulsador azul**, así que usar esos nombres lleva a conectar la entrada equivocada.

El pulsador es un contacto mecánico y **rebota**: al cerrarse produce durante unos milisegundos una ráfaga de transiciones. Si cuentas flancos sin filtrar, una sola pulsación se convierte en varias órdenes y la puerta hace cosas absurdas. Tendrás que filtrarlo, y el apartado 6 te dice cómo hacerlo sin bloquear.

4.3 Salidas

SALIDA	SOPORTE	QUÉ REPRESENTA
motorAbrir / motorCerrar	Dos variables lógicas	Las dos órdenes del motor. **Las dos a 0 significa motor parado**, y por tanto puerta quieta. Nunca pueden estar las dos a 1 al mismo tiempo.
Barra de posición	LED 1 a LED 8 del módulo	Cuántos LEDs encendidos, más abierta está la puerta: 0 LEDs con la puerta cerrada, 8 con la puerta abierta.
Display	Los 8 dígitos de siete segmentos	El estado en reposo, o la cuenta atrás en segundos mientras la puerta se mueve.

Que las dos órdenes del motor sean variables y no LEDs es deliberado: son la **salida real** de la máquina de estados, y en un equipo de verdad irían a un contactor. En la memoria tendrás que justificar en qué estados vale 1 cada una.

5. Comportamiento exigido

5.1 Estados

ESTADO	SITUACIÓN	DISPLAY	MOTOR
CERRADO	Hoja abajo, posicion = 0	cErrAdA	parado

ESTADO	SITUACIÓN	DISPLAY	MOTOR
ABRIENDO	Hoja subiendo	cuenta atrás	abrir
ABIERTO	Hoja arriba, posicion = 300	AbiErtA	parado
CERRANDO	Hoja bajando	cuenta atrás, o bArrErA si hay obstáculo	cerrar
PARADO_ABRIENDO	Detenida a mitad; venía abriendo	PArAdA	parado
PARADO_CERRANDO	Detenida a mitad; venía cerrando	PArAdA	parado

> Hacen falta **dos** estados de parada, no uno. El enunciado exige que la puerta invierta la marcha, así que el sistema tiene que recordar en qué sentido iba cuando se detuvo. Con un único estado PARADO esa información se pierde y el requisito no se puede cumplir.

5.2 Transiciones

La única entrada que provoca transiciones es el flanco de pulsación del MANDO, salvo los dos finales de recorrido.

ESTADO ACTUAL	SUCESO	ESTADO SIGUIENTE
CERRADO	pulsación de MANDO	ABRIENDO
ABIERTO	pulsación de MANDO	CERRANDO
ABRIENDO	pulsación de MANDO	PARADO_ABRIENDO
ABRIENDO	posicion alcanza 300 cm	ABIERTO

ESTADO ACTUAL	SUCESO	ESTADO SIGUIENTE
CERRANDO	pulsación de MANDO	PARADO_CERRANDO
CERRANDO	posicion alcanza 0 cm	CERRADO
PARADO_ABRIENDO	pulsación de MANDO	CERRANDO
PARADO_CERRANDO	pulsación de MANDO	ABRIENDO

5.3 La fotocélula

La fotocélula **solo actúa durante el cierre**. Es lo que hace una instalación real: al abrirse, la puerta se aleja del obstáculo y no hay peligro; al cerrarse, lo aplasta.

- Mientras haya obstáculo y el estado sea CERRANDO, la orden **motorCerrar pasa a 0** y la hoja deja de moverse de inmediato.
- El sistema **no cambia de estado**: sigue en CERRANDO, esperando. En el display aparece bArrErA.
- En cuanto desaparece el obstáculo, el cierre **se reanuda solo**, sin necesidad de tocar el mando.
- Durante la apertura la fotocélula se ignora por completo.

6. Desarrollo paso a paso

Trabaja en este orden y comprueba cada paso en la placa antes de seguir con el siguiente. Un paso que no funciona contamina todos los que vienen detrás.

Paso 1. Proyecto y biblioteca

Crea el proyecto en STM32CubeMX para la NUCLEO-F401RE e impórtalo en STM32CubeIDE, igual que en el Paso 1 de la P1 —con **STM32CubeIDE** como Toolchain/IDE— . Configura en CubeMX los tres pines del módulo como salidas (CLK, STB, DIO), genera,

añade TM1638.c y TM1638.h a Core/Src e inicialízalo. El anexo A tiene la referencia de la biblioteca y el anexo B el conexionado.

> Si usas **sprintf** o **snprintf**, incluye <stdio.h> en el bloque USER CODE Includes. Sin esa línea el compilador de STM32CubeIDE 2.x **da error**, no aviso, y el proyecto no construye.
>
> **Guarda el proyecto en una carpeta sin letras acentuadas.** Si la ruta lleva alguna —una carpeta llamada «Práctica 2», o un usuario que se llame «José»— el programa compilará y se grabará en la placa sin problema, pero **al depurar CubeIDE no encontrará el main.c** y solo te ofrecerá ver ensamblador. El compilador guarda la ruta dentro del ejecutable en UTF-8 y el depurador la lee con la codificación de Windows: para cualquier letra acentuada las dos no coinciden. Los espacios y los paréntesis no molestan; solo las tildes y la eñe. Si ya te ha ocurrido, mueve el proyecto a una ruta limpia y recompila desde cero con Project → Clean.

Paso 2. Base de tiempo de 100 ms

Antes de tocar la máquina de estados, consigue un pulso periódico fiable. La forma correcta es comparar HAL_GetTick() contra una marca previa, y avanzar la marca sumándole el periodo:

BASE DE TIEMPO SIN DERIVA

```
static uint32_t tMov = 0;

bool tick = (HAL_GetTick() - tMov) >= TICK_MOV_MS;
if (tick) {
    tMov += TICK_MOV_MS;   /* sumar el periodo, NO asignar HAL_GetTick()
*/
}
```

Si en lugar de sumar el periodo asignases tMov = HAL_GetTick(), cada vuelta perdería los microsegundos que el bucle tarda en llegar hasta ahí, y el error se acumularía: al cabo de un minuto el reloj iría notablemente atrasado. Explica esta diferencia en la memoria.

Paso 3. Flanco del mando, con antirrebote no bloqueante

Necesitas detectar el instante en que el pulsador pasa a pulsado, una sola vez por pulsación. La técnica es esperar a que la lectura lleve estable un tiempo mínimo antes de darla por buena:

ANTIRREBOTE POR VENTANA DE ESTABILIDAD

```
static bool     estable = false;   /* estado ya validado   */
static bool     ultimo  = false;   /* ultima lectura cruda */
static uint32_t tCambio = 0;

bool crudo = (HAL_GPIO_ReadPin(B1_GPIO_Port, B1_Pin) == GPIO_PIN_RESET);

if (crudo != ultimo) {             /* la senal se movio: reinicia la
ventana */
    ultimo  = crudo;
    tCambio = HAL_GetTick();
}
if ((HAL_GetTick() - tCambio) >= ANTIRREBOTE_MS && crudo != estable) {
    estable = crudo;
    return estable;                /* true solo al pulsar, no al soltar
*/
}
return false;
```

> **Prohibido usar HAL_Delay para el antirrebote.** Durante esa espera el programa no lee la fotocélula ni mueve la puerta: el sistema se queda ciego. Un HAL_Delay(500) con pasos de 100 ms pierde cinco pasos de movimiento. Cualquier entrega que use HAL_Delay dentro del bucle principal suspende el criterio 5 del baremo.

Paso 4. La máquina de estados

Escribe el bucle principal en cinco bloques, siempre en este orden. Mantenerlos separados es lo que hace que el programa se pueda leer y depurar:

1. **Leer las entradas** una sola vez por vuelta y guardarlas en variables locales.
2. **Transiciones**: un switch sobre el estado que solo decide cuál será el estado siguiente. Aquí no se toca nada más.

3. **Órdenes del motor**: calcular motorAbrir y motorCerrar a partir del estado y del obstáculo.
4. **Movimiento**: si toca tick y el motor está en marcha, actualizar posicion.
5. **Visualización**: display y barra de LEDs, según el estado y la posición.

La tentación es mezclar la decisión con la acción dentro del mismo switch. Resiste: en cuanto el sistema crece, ese código se vuelve imposible de seguir.

Paso 5. Visualización

La barra de LEDs se obtiene escalando la posición al número de LEDs, y la cuenta atrás dividiendo los centímetros que faltan entre la velocidad. Los dos son cálculos con enteros: cuida el orden de las operaciones para no perder precisión.

ESCALADO DE LA BARRA

```
uint8_t encendidos = (pos_cm * N_LEDS) / RECORRIDO_CM;   /* multiplicar
ANTES de dividir */
```

Paso 6. Mira la máquina de estados moverse

Una máquina de estados es fácil de escribir y difícil de comprobar: desde fuera solo se ve la puerta. Con el depurador se ve el estado.

6. Pon un **punto de ruptura condicional** en el bucle principal, con la condición `estado == stParadoAbriendo || estado == stParadoCerrando`: son los dos estados de parada que declara el `typedef enum` del proyecto.
7. Provoca esa transición con el pulsador y comprueba que el programa se detiene ahí, y solo ahí.
8. Inspecciona el estado y la posición: ¿son los que esperabas en ese instante?

> Un punto de ruptura **sin condición** en esa línea es inservible: para en cada transición y acabas pulsando «continuar» veinte veces. La condición es lo que convierte la herramienta en útil.

Aprovecha para comprobar una cosa más: pon un punto de ruptura en el antirrebote y observa **cuántas veces se ejecuta** por cada pulsación tuya. Anótalo en la memoria.

7. Criterios de aceptación

Tu programa se comprobará con estas seis pruebas, en la placa. Están redactadas para que puedas pasarlas tú mismo antes de entregar. Si alguna falla, todavía no has terminado.

Nº	PRUEBA	RESULTADO EXIGIDO
CA-1	Desde la puerta cerrada, una pulsación de MANDO y se deja correr.	La puerta abre por completo. El recorrido dura **10,0 s ± 0,3 s** y termina con los 8 LEDs encendidos y AbiErtA en el display.

Nº	PRUEBA	RESULTADO EXIGIDO
CA-2	Durante la apertura, pulsar MANDO a mitad de recorrido; después, pulsar otra vez.	La puerta se detiene en el acto y muestra PArAdA. La segunda pulsación la pone a **cerrar**, no a abrir.
CA-3	Durante el cierre, mantener pulsada S1 unos segundos y después soltarla.	La hoja se detiene mientras S1 está pulsada, el display muestra bArrErA y la posición no cambia. Al soltar, **el cierre se reanuda solo**.
CA-4	Durante la apertura, mantener pulsada S1.	La apertura continúa con normalidad: la fotocélula no interviene al abrir.
CA-5	Observar la barra de LEDs a lo largo de un recorrido completo.	Progresa de 0 a 8 LEDs de forma monótona, y a mitad de recorrido hay exactamente **4 LEDs** encendidos.
CA-6	Observar el display durante un recorrido completo.	La cuenta atrás arranca en 10.0, decrece de forma continua y **nunca marca 0.0 con la puerta todavía en movimiento**.

8. Criterios de evaluación

> **Condición previa.** El proyecto entregado debe compilar en STM32CubeIDE **sin errores y sin avisos**. Un proyecto que no compila no se evalúa: se devuelve para corrección y se califica sobre 5.

La práctica se califica sobre 10 puntos con el siguiente baremo. Cada criterio se valora en tres niveles: no alcanzado (0 puntos), alcanzado a medias (mitad del peso) y alcanzado (peso completo).

CRITERIO	QUÉ SE VALORA	ALCANZADO A MEDIAS	PESO
1. Recorrido y temporización	CA-1: la puerta completa el recorrido en el tiempo previsto.	Abre y cierra, pero el tiempo se desvía más de 0,3 s del exigido.	2,0
2. Parada e inversión	CA-2: la pulsación detiene el movimiento y la siguiente invierte el sentido.	Detiene correctamente, pero al reanudar sigue en el mismo sentido.	2,0
3. Fotocélula	CA-3 y CA-4: detiene el cierre, reanuda al liberar y no afecta a la apertura.	Detiene el cierre pero exige tocar el mando para reanudar, o interviene también al abrir.	2,0
4. Visualización	CA-5 y CA-6: barra de LEDs y cuenta atrás correctas.	Uno de los dos elementos es correcto y el otro no.	1,5
5. Calidad del código	Bucle no bloqueante, sin HAL_Delay; transiciones y acciones separadas; constantes con nombre en lugar de números sueltos.	Funciona y está ordenado, pero queda algún HAL_Delay o algún número mágico.	1,5
6. Diagrama y memoria	Diagrama de estados coherente con el código entregado y justificación de las dos cuestiones del apartado 9.	Diagrama correcto pero sin justificar, o justificación sin diagrama.	1,0

9. Entregables

- **Proyecto de STM32CubeIDE** comprimido en un único archivo, **sin la carpeta Debug**. Nombre: ACII_P2_Apellido1_Apellido2_Nombre.zip
- **Diagrama de estados** con los seis estados y las ocho transiciones de la tabla del apartado 5.2. Vale a mano y fotografiado, siempre que se lea.
- **Memoria breve**, máximo dos páginas, que responda a estas dos preguntas:
    - Por qué hacen falta dos estados de parada y no uno.
    - Qué diferencia hay entre avanzar la marca de tiempo sumándole el periodo y reasignarle HAL_GetTick(), y qué efecto tiene a lo largo de varios minutos.

Entrega por Moovi en el plazo indicado en la ficha. La práctica se defiende en el laboratorio: se te pedirá ejecutar los criterios de aceptación sobre tu propia placa y explicar cualquier parte del código.

10. Uso de asistentes de inteligencia artificial

Se permite el uso de asistentes de IA como herramienta de consulta y de depuración, con tres condiciones:

- **Declararlo** en la memoria: qué asistente y para qué lo has usado.
- **Entender lo que entregas.** En la defensa se te puede pedir que expliques cualquier línea del código o que la modifiques en el momento. No saber explicar el propio código equivale a no haberlo entregado.
- **No sustituir el trabajo de diseño.** El diagrama de estados y las respuestas de la memoria son tuyos.

Anexo A. La biblioteca TM1638

El módulo LED&KEY se gobierna con TM1638.c y TM1638.h, que se te entregan ya escritos. No hay que modificarlos.

A.1 Inicialización

```
/* USER CODE BEGIN 0 */
TM1638 TM;
/* USER CODE END 0 */

/* USER CODE BEGIN 2 */
TM.clk_port = CLK_GPIO_Port;  TM.clk_pin = CLK_Pin;
TM.stb_port = STB_GPIO_Port;  TM.stb_pin = STB_Pin;
TM.dio_port = DIO_GPIO_Port;  TM.dio_pin = DIO_Pin;

tm1638_TurnOn(&TM, 7);        /* brillo maximo */
tm1638_DisplayClear(&TM);
/* USER CODE END 2 */
```

A.2 Funciones

FUNCIÓN	QUÉ HACE
tm1638_TurnOn(&TM, brillo)	Enciende el display. brillo de 0 a 7.
tm1638_DisplayClear(&TM)	Apaga todos los dígitos.
tm1638_Led(&TM, n, on)	Enciende (1) o apaga (0) el LED n, de 1 a 8.
tm1638_DisplayTxt(&TM, "texto")	Escribe una cadena, alineada a la derecha. Un punto se une al dígito anterior.
tm1638_DisplayChar(&TM, n, c, punto)	Escribe un carácter en la posición n.

FUNCIÓN	QUÉ HACE
tm1638_ScanButtons(&TM)	Lee de una vez el estado de las ocho teclas y lo devuelve en un byte.
tm1638_KeyState(&TM, teclas, n)	Sobre el byte anterior, indica si la tecla n está pulsada.
tm1638_ReadKey(&TM, teclas)	Sobre el byte anterior, devuelve el número de la tecla pulsada.

> El display **no puede representar cualquier letra**: son siete segmentos. Están disponibles las cifras, el espacio, A B C E F H I J L O P S U, y a b c d h i n o r t u y. Una letra fuera de esa lista aparece como un dígito en blanco o con basura. Comprueba tus cadenas antes de darlas por buenas.

Para leer las teclas, el patrón correcto es escanear una vez por vuelta de bucle y consultar después ese byte tantas veces como haga falta, en lugar de escanear una vez por cada consulta:

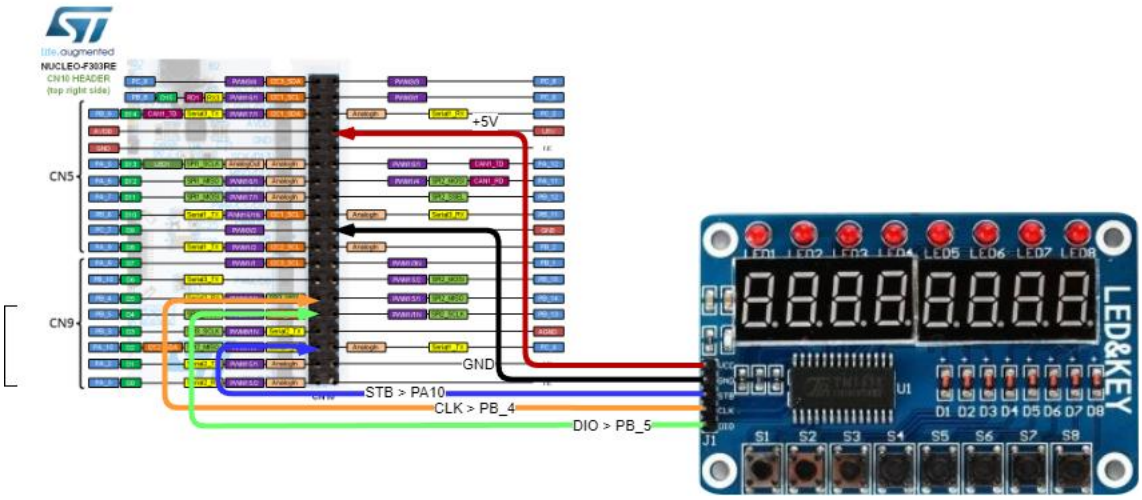
```
uint8_t teclas = tm1638_ScanButtons(&TM);
bool obstaculo = tm1638_KeyState(&TM, teclas, 1);   /* S1 = fotocelula
*/
```

Anexo B. Conexionado

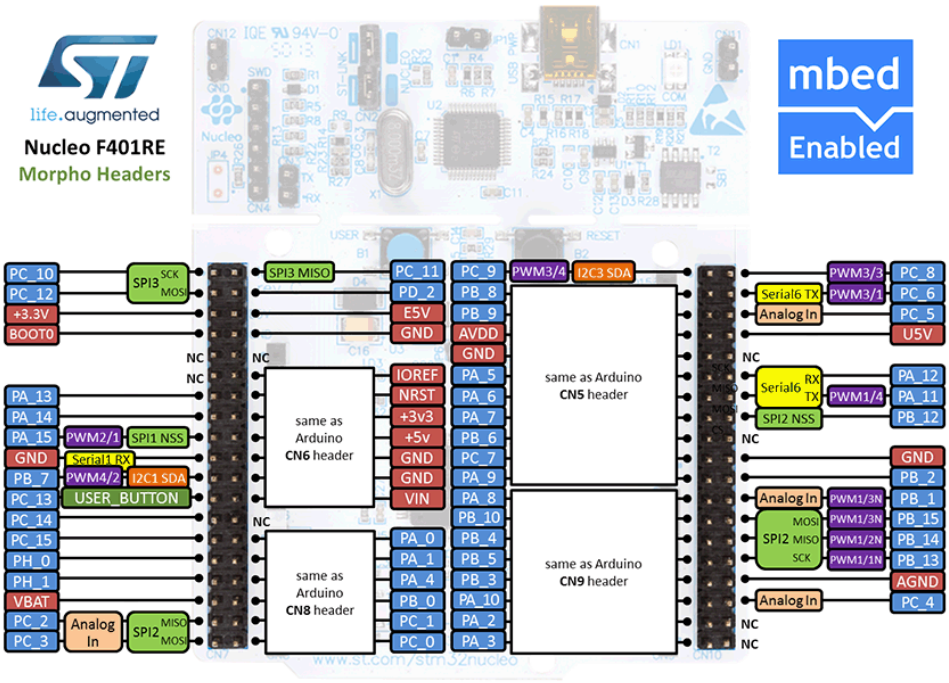
El módulo se alimenta a 3,3 V desde la propia placa y usa tres líneas de datos. Esta es la asignación de pines del proyecto de referencia; declara estas mismas etiquetas en CubeMX y los nombres CLK_Pin, STB_Pin y DIO_Pin aparecerán solos en main.h.

SEÑAL	PIN DEL STM32	ETIQUETA EN CUBEMX	MODO
STB	PA10	STB	GPIO_Output
CLK	PB4	CLK	GPIO_Output
DIO	PB5	DIO	GPIO_Output
MANDO	PC13	B1	Entrada (pulsador azul de la placa)

> La línea **DIO es bidireccional**: la biblioteca la conmuta a entrada por su cuenta cuando toca leer las teclas y la devuelve a salida al terminar. Por eso en CubeMX se declara como salida y aun así funciona la lectura. En esta práctica el MANDO se lee **por consulta**, dentro del bucle; la versión por interrupción llega en la Práctica 4.



CONEXIÓN DEL MÓDULO LED&KEY A LA NUCLEO-F401RE.

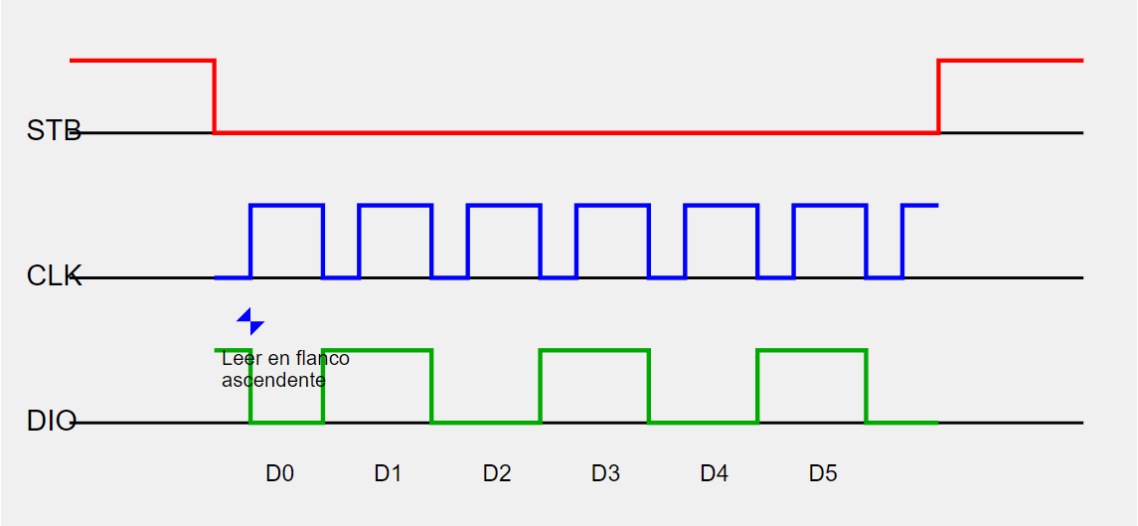


PINOUT DE LOS CONECTORES MORPHO DE LA NUCLEO-F401RE, PARA LOCALIZAR PA10, PB4 Y PB5.

Anexo C. El protocolo del TM1638

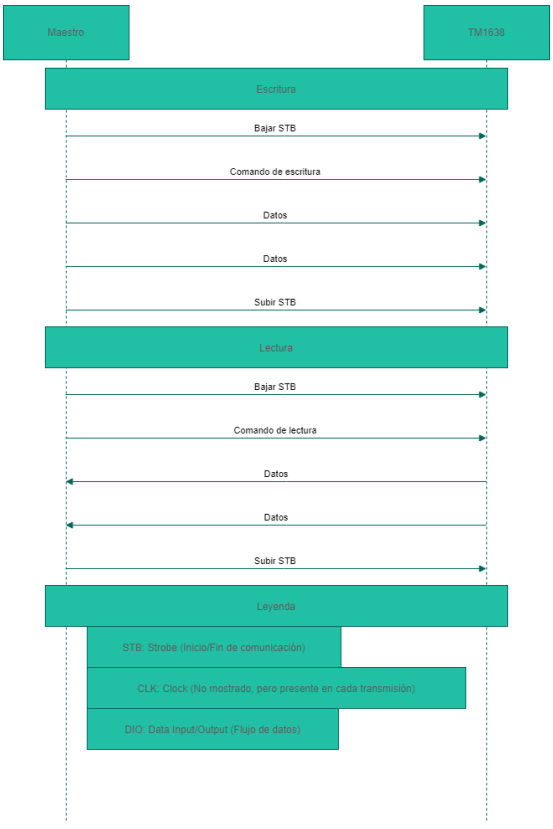
No necesitas implementarlo —la biblioteca ya lo hace—, pero sí entenderlo: es un protocolo serie síncrono de tres hilos, y aparece en el examen.

- **STB (Strobe)** delimita la trama. El maestro lo baja para empezar y lo sube para terminar; al subirlo, el TM1638 procesa lo recibido.
- **CLK (Reloj)** marca cada bit. El dato se lee en el **flanco de subida**.
- **DIO (Datos)** transporta los bits, en grupos de 8 y **empezando por el menos significativo**.
- El primer byte tras bajar STB es siempre un **comando**; los siguientes son datos.



CRONOGRAMA DE UNA TRAMA: STB EN ROJO, CLK EN AZUL, DIO EN VERDE. EL DATO SE LEE EN EL FLANCO ASCENDENTE DEL RELOJ. EL DIBUJO MUESTRA SEIS BITS POR CLARIDAD; EL TM1638 TRABAJA CON BYTES COMPLETOS.

La diferencia entre escribir y leer está en el comando inicial y en quién gobierna la línea DIO. Al escribir, la controla el maestro de principio a fin. Al leer, después del comando el microcontrolador conmuta su pin DIO a entrada y es el TM1638 el que emite.



SECUENCIA COMPLETA DE UNA ESCRITURA Y DE UNA LECTURA.